

Model Momentum III — Implementation Guide

Module Interfaces, Data Contracts, Build Order, and the Phase 0 Gate

Abstract

This is the build specification for the information-geometric momentum predictor of thought anchors. It is the companion to the design document (“Model Momentum III”); that document fixes *what* the method is and *why*, and is assumed here without re-derivation. This guide fixes the *how*: the module boundaries, the on-disk data contracts, the one architectural decision (forward-pass memory) that everything organizes around, default values for every free knob, and — most importantly — the order in which to build and run, with cheap empirical checks gating expensive compute. The guiding principle is **design holistically, build incrementally**: all interfaces and schemas are specified up front so early plumbing anticipates late plumbing, but modules are built and run in an order that surfaces the unknown-by-reading questions (tokenizer/stop-set match, alignment round-trip, scale validity, signal-vs-baseline) before any gold-level labeling cost is paid. No production code is given; each component is specified precisely enough to implement directly.

Contents

1	How to Use This Document	1
2	Repository Architecture	1
3	The One Architectural Decision: Forward-Pass Memory	2
4	Data Contracts (Schemas)	3
4.1	Trace record (produced by <code>generate.py</code>)	3
4.2	Span record (produced by <code>segment.py</code> , stored with the trace)	3
4.3	Label record (produced by <code>generate.py</code>)	3
4.4	Score record (produced by <code>momentum.py</code>)	4
5	Module Interfaces	4
5.1	<code>config.py</code>	4
5.2	<code>segment.py</code>	5
5.3	<code>generate.py</code>	5
5.4	<code>model.py</code>	5
5.5	<code>momentum.py</code>	6
5.6	<code>evaluate.py</code>	6
6	Knobs and Default Values	7
7	Build Order	7

8 The Phase 0 Gate	9
8.1 Two-phase similarity median (sequencing constraint)	9
9 Deferred-by-Design: Stub Now, Build If Triggered	9
10 Cross-Cutting Correctness Requirements	10
11 Flat Build Checklist	10

1 How to Use This Document

Read Sections 2–6 (architecture, the memory decision, schemas, interfaces, knobs) *before writing any code* — they are the holistic design layer and they are coupled, so deciding them piecemeal causes rework. Then follow Section 7 (build order) literally; it is sequenced so that the cheapest checks run first and the single expensive step (gold labeling) sits behind a gate (Section 8). Section 9 lists the pieces to stub now and implement only if a test triggers them. Section 11 is a flat checklist to track against.

2 Repository Architecture

Six modules, one directional data flow. Each module’s output is another’s input through a file contract (Section 4), never through shared memory, so any stage can be re-run from stored artifacts without recomputing upstream.

Module	Responsibility	Consumes / Produces
<code>config.py</code>	all knobs, paths, model id, seeds	— / config object
<code>segment.py</code>	two-stage boundary-respecting spans	trace text / span table
<code>generate.py</code>	base traces + resample-and-filter labels	problems / labeled trace store
<code>model.py</code>	teacher-forced logits, streaming	stored token ids / per-step logit access
<code>momentum.py</code>	the geometry: F, R, C per span	logits + spans / score table
<code>evaluate.py</code>	partial Spearman, bootstrap, baselines	scores + labels / results
<code>tests/</code>	synthetic-logit geometry checks	— / pass/fail

Data flow: **generate** (uses **segment** internally) → labeled trace store → **model** re-forwards stored ids → **momentum** scores → **evaluate** correlates scores against stored labels. **segment** is also called standalone by **generate**; it is factored out because both generation and any re-segmentation must use the *identical* boundary logic.

Single source of truth for boundaries and the content mask. The clause/sentence parser (**segment**) and the content-token stop-set (**momentum**) both depend on the same notion of “structural punctuation.” Define that set *once* in `config.py` (as token ids resolved against the actual tokenizer, not characters) and import it into both. If these drift apart, spans and scores silently disagree about what a boundary is.

3 The One Architectural Decision: Forward-Pass Memory

Everything organizes around how logits are obtained and held, so decide this first. The full logit tensor for one trace is $[L, V]$ with $L \sim$ thousands and $V \sim 10^5$; materializing it in fp32 is gigabytes and is the most likely cause of an out-of-memory failure. We adopt **teacher-force-and-stream**:

1. Run a single forward pass over the stored token-id sequence with caching, but **do not return all logits**. Instead, iterate positions in order and expose one logit vector at a time to the scorer.
2. The scorer (Section 5.5) holds only: the previous *content*-position scaled-logit vector $\mathbf{g}_{k-1}$, the EMA state $\bar{\mathbf{v}}$, and per-position scalar outputs. Peak extra memory is $O(V)$, not $O(LV)$.
3. This is byte-identical to the online form (during generation logits arrive one at a time), so offline scoring and online inference share one code path — a property to preserve, not an accident.

Why not chunk-and-cache logits to disk. Writing $[L, V]$ logits per trace to disk for later scoring is simpler to reason about but multiplies storage by V and breaks the offline/online equivalence. We store *token ids* (tiny) and re-forward when scoring; re-forwarding one trace is one cheap pass and avoids the storage blowup. The only quantity worth caching is the small per-span score table, not the logits.

Practical consequence for model.py. Its public surface is an iterator, not a tensor return. Conceptually: `stream_logits(token_ids) -> yields (position, scaled_logit_vector)`, with temperature scaling applied inside. The scorer consumes the stream; nothing assembles the full matrix.

4 Data Contracts (Schemas)

All inter-module artifacts are on disk with explicit schemas. Token ids are the source of truth; text and character offsets are derived. Use a columnar/record format that stores integer arrays natively (e.g. Parquet or JSONL-with-arrays); the field semantics below are what matter, not the container.

4.1 Trace record (produced by generate.py)

One record per base trace.

Field	Type	Meaning
trace_id	str	unique key
problem_id	str	source problem (for trace-level grouping)
token_ids	int[]	the generated sequence; source of truth
text	str	detokenization of <code>token_ids</code> (derived)
temperature	float	generation T (required for the metric)
think_start_tok	int	index of first token inside <code><think></code>
think_end_tok	int	index of first token after <code></think></code>
is_correct	bool	did this trace reach the verified answer
model_id	str	exact model+revision used

4.2 Span record (produced by `segment.py`, stored with the trace)

One record per span; spans tile the think region.

Field	Type	Meaning
<code>trace_id</code>	str	foreign key
<code>span_id</code>	int	ordinal within trace (also the position control)
<code>tok_start</code>	int	half-open start, token-id index
<code>tok_end</code>	int	half-open end, token-id index
<code>char_start</code>	int	derived, for cross-check only
<code>char_end</code>	int	derived, for cross-check only
<code>unit_id</code>	int	which parsed sentence/clause this window belongs to
<code>n_tokens</code>	int	<code>tok_end - tok_start</code>

4.3 Label record (produced by `generate.py`)

One record per span; the ground truth.

Field	Type	Meaning
<code>trace_id</code>	str	foreign key
<code>span_id</code>	int	foreign key
<code>label</code>	float	divergence between pools (KL default; TV optional)
<code>label_kind</code>	str	"kl" or "tv"
<code>n_real</code>	int	usable same-meaning continuations
<code>n_diff</code>	int	usable different-meaning continuations
<code>pool_filled</code>	bool	did both pools reach R (else span dropped)
<code>median_sim_used</code>	float	the dataset-median cosine threshold applied

4.4 Score record (produced by `momentum.py`)

One record per span; the predictor side. Computed both for the content-only default and the mixed-EMA ablation, distinguished by `variant`.

Field	Type	Meaning
<code>trace_id</code>	str	foreign key
<code>span_id</code>	int	foreign key
<code>variant</code>	str	"content_only" (default) or "mixed"
<code>force</code>	float	$\bar{F}_s$, mean Fisher force over scored steps
<code>redirection</code>	float	$R_s \in [0, 1]$, turning energy fraction
<code>sign</code>	float	C_s , energy-weighted projection (> 0 accel, < 0 brake)
<code>surprise</code>	float	pooled $-\log p(x)$ baseline
<code>n_scored</code>	int	content steps contributing (n_s)
<code>beta</code>	float	EMA decay used
<code>in_think</code>	bool	span lies in reasoning region (not answer emission)

Join discipline. Every downstream join is on (`trace_id`, `span_id`) against the *stored* span table — never by re-segmenting text. Before any correlation, assert that span counts per trace match across the span, label, and score tables; a mismatch is a bug, not a result.

5 Module Interfaces

For each module: its job, the functions to write (signatures as intent, not code), and the non-obvious correctness requirements. “Write `f(a) -> b`” means implement a function with that input/output contract.

5.1 `config.py`

Holds every knob (Section 6) and all paths in one place; no logic. Critically, it resolves and exposes the **structural-token id set** (the shared boundary/stop-set definition) against the loaded tokenizer, so `segment` and `momentum` import the same object. Also fixes seeds.

5.2 `segment.py`

Job: turn a trace’s token ids into the span table via the two-stage rule.

- `parse_units(token_ids, think_range) -> list[unit_span]`: stage 1, split into sentences/-clauses at structural tokens (from `config`). Operates on token ids; punctuation is identified by id, not by regex over text.
- `window_units(unit_spans, t, t_min) -> list[span]`: stage 2, sub-split any unit longer than t into $\leq t$ -token windows entirely inside the unit; merge a trailing fragment $< t_{\min}$ into the previous window of the same unit.
- Correctness: windows never cross a unit boundary; spans tile the think region with no gaps or overlaps; every span carries its `unit_id`.

5.3 `generate.py`

Job: produce base traces and the resample-and-filter labels. The expensive module; built only after the Phase 0 gate logic is in place. Sub-steps:

- `generate_base_traces(problems) -> trace records`: sample one CoT per problem at temperature T ; store token ids, think range, correctness. Verify correctness against the task’s gold answer.
- `resample_slot(trace, span, R_target, oversample_cap) -> continuations`: for the span’s slot, sample continuations *from the token just before `tok_start`* (regenerating the slot and everything after), up to `oversample_cap`, batched across GPUs. Each continuation yields a regenerated slot text and a final answer.
- `embed_and_split(orig_slot, continuations, median_sim) -> (real_pool, diff_pool)`: embed slots with the sentence embedder; same-meaning if cosine to original $> \text{median_sim}$, different if $<$. (See the two-phase median in Section 8.1.)

- `label_from_pools(real_pool, diff_pool, kind)` -> `label`: aggregate each pool's final-answer distribution; label is KL (default) or TV between them. Set `pool_filled=False` and drop the span if either pool $< R$.

Oversampling and pool-fill policy (must be explicit). The same/different split is post hoc, so some slots yield a lopsided split (a slot the model almost always regenerates near-identically gives few below-median samples). Policy: sample in batches up to `oversample_cap` total; stop early once both pools reach R ; if `oversample_cap` is hit with either pool $< R$, mark `pool_filled=False` and exclude the span from analysis (recorded, not silently dropped). Report the drop rate — a high rate at a given t is itself information about granularity.

5.4 model.py

Job: deterministic teacher-forced logit access, streaming (Section 3).

- `load_model(model_id)` -> `(model, tokenizer)`: fixed revision, eval mode, fixed dtype.
- `stream_logits(token_ids, T)` -> `iterator of (pos, scaled_logit_vec)`: one forward pass; yields scaled logits ($g = \ell/T$) one position at a time; never returns the full tensor. Casts to fp32 for the vector handed out (the covariance reduction needs it).
- Correctness: indexing convention is fixed — `logits[i]` predicts token $i+1$; teacher forcing must reproduce the *stored* ids exactly (no re-tokenization of text). Assert the input ids round-trip.

5.5 momentum.py

Job: the geometry. Consumes the logit stream and the span table; produces the score table. This is the module to build *first* (against synthetic logits), because it has no model or data dependency beyond the stream interface.

- `fisher_cov(p, u, w)` -> `scalar`: the exact categorical Fisher inner product $\sum puw - (\sum pu)(\sum pw)$, fp32.
- `is_content(token_id)` -> `bool`: true unless the id is in the structural/special stop-set from `config`.
- `score_stream(logit_stream, is_content, beta, variant)` -> `per-position (F, perp, tot, cproj)`: the streaming loop. `content_only` variant (default): advance the trajectory only across content positions, EMA updates only on content steps, velocity is content-to-content. `mixed` variant (ablation): advance through every token, score only content. Base point is the previous content distribution p_{k-1} ; force is $\sqrt{\text{cov}(d, \bar{d})}$ with $d = v - \bar{v}$.
- `pool_to_spans(per_position, spans, eps)` -> `score records`: $\bar{F}_s$ = mean force over content steps in the span; $R_s = \sum \text{perp} / \sum \text{tot}$; C_s = energy-weighted mean of `cproj`. Emit NaN and drop if $n_s = 0$.
- Correctness: EMA warmup means the first span per trace is unreliable — flag it for dropping in `evaluate`, do not special-case it here.

5.6 evaluate.py

Job: turn scores + labels into the headline numbers and the test verdicts.

- `partial_spearman(pred, label, covars)` -> `rho`: rank-transform all; residualize `label` and `pred` on {span index, span length} by OLS; Pearson-correlate residuals.
- `trace_bootstrap(table, statistic_fn, n=1000)` -> (`point`, `ci_low`, `ci_high`): resample *traces* with replacement (never spans); recompute the statistic each draw; report 2.5/97.5 percentiles.
- `run_tests(scores, labels)` -> `verdicts`: compute Test 1–6 quantities (redirection partial- ρ , sign/taxonomy enrichment, boundary-clustering of force, transfer, blind-spot fraction, β curve) and emit the pre-registered verdict per the design document.
- Correctness: filter to reasoning-region spans (`in_think`), drop first-span-per-trace, drop `pool_filled=False`; always report against both `content_only` and `mixed` so the ablation gap is visible.

6 Knobs and Default Values

Every free parameter, with a recommended default to start from. All live in `config.py`.

Knob	Default	Notes
β (EMA decay)	0.97	memory $\approx 1/(1 - \beta)$ <i>content</i> tokens; swept in Test 6
t (window budget)	15 tokens	<i>budget-dominating</i> ; calibrate at Phase 0 over {10, 15, 25} by label stability / drop rate
$t_{\min}$ (merge floor)	5 tokens	trailing fragments below this merge left
motion threshold	small > 0	gate C_s where $\langle v, v \rangle$ is negligible; calibrate in Phase 0
ε	10^{-8}	denominator floor
R (per pool)	100	<i>budget-dominating</i> ; “gold” level, but confirm sufficiency at Phase 0
oversample cap	$\sim 8 \times R$	ceiling on total rollouts per slot before giving up
similarity threshold	dataset median	parameter-free; computed in two phases (Sec. 8.1)
embedder	Sentence-BERT	or a current sentence embedder; fixed per study
label kind	KL	TV optional; record which
temperature T	match gen. run	the metric must use the sampling distribution
answer-region cutoff	after <code></think></code>	token-id test, not heuristic
bootstrap draws	1000	trace-level
dtype (forward)	bf16/fp16	covariances always fp32

t and R are budget-dominating knobs — calibrate them at Phase 0, do not fix them up front. Most knobs above are set once and forgotten. Two are different: the window budget t and the per-pool rollout count R together determine the dominant compute cost of the whole study, because total rollouts scale roughly as (number of spans) $\times$ (rollouts per span) — smaller t means more spans, larger R means more rollouts each. The defaults ($t = 15$, $R = 100$) are starting points chosen to match the sources, not validated optima. Treat them as *Phase-0-calibrated*: the build order (Step 4 and the gate) deliberately measures label stability and pool-fill drop rate across $t \in \{10, 15, 25\}$ on the tiny set *before* the gold run, so you pick the granularity that localizes anchors without an unacceptable drop rate and confirm R is large enough that labels are stable, before paying for it at full scale. Committing to t and R up front is the most likely way to either overspend or generate a gold dataset at the wrong granularity. This is a judgement call, flagged as such: the numbers are provisional, and Phase 0 exists in part to set them.

7 Build Order

Build in this order. The principle: validate the math with no compute, then the plumbing with one cheap forward pass, then the full loop on a handful of traces, and only then pay for gold labeling. Each step has an explicit exit criterion.

Step 0 — config.py + skeleton. Lay down the repo, the schemas as typed records, the config with defaults, and the shared structural-token set resolved against the tokenizer. *Exit*: importing any module and reading config works; the stop-set prints as actual token ids.

Step 1 — momentum.py against synthetic logits. Build the geometry and the streaming loop with *no model and no data*. Feed hand-constructed logit streams from `tests/`. *Exit* (*unit tests pass*):

- `fisher_cov` matches a brute-force covariance on random vectors; the all-ones gauge vector gives ≈ 0 .
- A straight-line (constant-velocity) logit trajectory yields force ≈ 0 (geodesic $\Rightarrow$ no force).
- A trajectory that reverses direction yields $C_s < 0$; a sharp turn yields high R_s ; a speed-up along a fixed direction yields $C_s > 0$, low R_s .
- Content masking: inserting “formatting” positions changes nothing in the `content_only` variant.

This step is seconds of compute and catches every math/indexing error before a model is ever loaded.

Step 2 — model.py + single-trace alignment check. Load the model; generate or hand-pick *one* trace; store its ids; re-forward and stream. Run `segment` on it and `momentum` over it. *Exit* (*the highest-value cheap check*):

- Stored token ids round-trip: `detokenize` $\rightarrow$ `retokenize` equals the stored ids (or the stream is driven by ids directly, sidestepping this).

- The content stop-set actually fires on this tokenizer’s punctuation/newline tokens (inspect a few positions by hand).
- Spans tile the think region; `tok.start/tok.end` align to real clause boundaries on visual inspection.
- Scores are finite, the first span is flagged, the answer region is correctly excluded.

Cost: one forward pass. This is where tokenizer surprises surface; do not proceed until it is clean.

Step 3 — `evaluate.py` against mock labels. Build the stats harness and test it on *synthetic* labels (e.g. a known correlation injected) so the partial-Spearman and trace-bootstrap are verified independently of real data. *Exit*: recovers a planted correlation; trace-bootstrap CIs widen correctly when traces (not spans) are resampled.

Step 4 — `generate.py` + Phase 0 mini-run. Now build the expensive module, but run it small: a handful of traces ($\sim 10\text{--}20$) on the easiest task family, full resample-and-filter at $R = 100$. Run the whole pipeline end to end, and use this set to **calibrate the budget-dominating knobs**: re-segment at each $t \in \{10, 15, 25\}$ and compare label stability and pool-fill drop rate, and check that labels are stable at $R = 100$ (i.e. that a smaller R would not have sufficed, or that 100 is not excessive). Freeze t and R at the values this sweep justifies before any gold run. *Exit*: the pipeline completes, emits a score+label table with sane pool-fill rates, and t/R are chosen from data rather than assumed.

Step 5 — THE GATE (Section 8). Look at the Phase 0 numbers before scaling. Decision point, not a build step.

Step 6 — Gold run. Only if the gate passes: scale **generate** to the full dataset and curriculum, run the gold labeling, and produce the full results and Test 1–6 verdicts.

8 The Phase 0 Gate

Do not spend gold-level compute until Phase 0 clears this gate. After Step 4, on the tiny labeled set, check:

1. **Plumbing sanity**: joins line up, pool-fill rate is acceptable at the chosen t , no NaN floods, answer region excluded. **Also fix t and R here**: compare label stability and drop rate across $t \in \{10, 15, 25\}$ and confirm R is large enough that the labels are stable; these values are then frozen for the gold run.
2. **Direction of signal**: is the partial Spearman of R_s (or C_s) with the label *positive and plausibly nonzero*, even if not yet significant at this tiny n ? A near-zero or negative central estimate across the whole small set is a warning.
3. **Baseline posture**: does the geometry at least track alongside the surprise baseline rather than being dominated by it?

Pass → Step 6 (gold). **Fail on plumbing** → fix and re-run Phase 0 (cheap). **Fail on signal with clean plumbing** → this is a real result: invoke the design document’s pre-registered

responses (e.g. Test 3 boundary fix if force clusters at span starts; reconsider scale per the model-scale caveat) *before* paying for gold. Do not scale a flat signal hoping n fixes it.

8.1 Two-phase similarity median (sequencing constraint)

The “different-meaning” threshold is the dataset-median pairwise cosine similarity, so it *cannot be a constant known up front*. Sequence it: (phase A) generate base traces and all resample continuations, embedding and storing similarities but *not* yet splitting; (phase B) compute the global median over all collected similarities, then apply it to split every slot’s pool and compute labels. In the Phase 0 mini-run, compute a provisional median on the small set; recompute the real median on the gold set before gold labeling. Record `median_sim_used` per label so any later re-split is reproducible.

9 Deferred-by-Design: Stub Now, Build If Triggered

Write these as named, documented stubs so the interfaces exist, but do not implement until their trigger fires. Implementing them up front is speculative work the Phase 0/test results may tell you to skip.

- **Boundary fix for the small-step assumption** (Test 3 trigger: force clusters at span-initial tokens): EMA reset/down-weight at parsed boundaries, or drop the first k post-boundary content tokens. Stub: a no-op hook in the scoring loop at boundary positions.
- **Arc-length reparameterization** (Test 6 ablation): recompute with Fisher-arc-length time instead of the token clock. Stub: a clock-selection flag in the scorer.
- **Paraphrase perturbation route** (trigger: model too large / rollout budget too high for resample-and-filter): controlled meaning-changed rewrites in place of resampling. Stub: an alternative `make_alternative` behind the same interface `generate` already calls; must re-validate against a resample-and-filter subset before its labels are trusted.

10 Cross-Cutting Correctness Requirements

A short list of invariants every module must respect, gathered so they are not buried:

- **Token ids are truth.** Never reconstruct alignment from text; derive text/offsets from ids, never the reverse.
- **Determinism.** Fixed model revision, fixed seeds, eval mode; teacher forcing must reproduce stored ids exactly.
- **fp32 covariances.** Regardless of forward dtype.
- **Trace is the unit.** Every split, bootstrap, and significance statement groups by trace, never by span.
- **Both variants always.** Score `content_only` and `mixed` every run; the gap is a reported diagnostic.
- **Record, don’t silently drop.** Dropped spans (pool unfilled, $n_s = 0$, first-span warmup, answer region) are flagged in the tables, not removed upstream.

- **One boundary definition.** Parser and content mask import the same structural-token set.

11 Flat Build Checklist

- ☐ `config.py`: knobs, paths, seeds, shared structural-token id set resolved against tokenizer.
- ☐ Schemas: trace, span, label, score records as typed structures with the join keys.
- ☐ `momentum.py`: `fisher_cov`, `is_content`, `score_stream` (both variants), `pool_to_spans`.
- ☐ `tests/`: synthetic-logit geometry suite; *all pass* before Step 2.
- ☐ `model.py`: `load_model`, streaming `stream_logits`; id round-trip assertion.
- ☐ Single-trace alignment check clean (stop-set fires, spans tile, scores finite).
- ☐ `segment.py`: `parse_units`, `window_units`; tiling/no-cross invariants.
- ☐ `evaluate.py`: `partial_spearman`, `trace_bootstrap`, `run_tests`; verified on planted correlation.
- ☐ `generate.py`: `base_traces`, `resample_slot`, `embed_and_split` (two-phase median), `label_from_pools`, `oversample/pool-fill` policy.
- ☐ Phase 0 mini-run end to end (~ 10 – 20 traces, $R = 100$).
- ☐ **GATE**: plumbing sane, signal direction positive, baseline posture acceptable.
- ☐ Gold run + Test 1–6 verdicts (only past the gate).
- ☐ Stubs in place (boundary fix, arc-length, paraphrase) — unbuilt until triggered.